

Loading and Handling Data in R

Data Analytics using R – Chapter 3

Anil

2026-07-29

Challenges of Analytical Data Processing

Data Formats

- Data comes from varied sources (CSV, Excel, SQL, web, etc.)
- R supports many formats via packages (ODBC, readr, etc.)

Data Quality

- Missing values (NA), outliers, inconsistent data
- R provides functions to clean and impute

Project Scope

- Time, cost, and data volume must be planned

Stakeholder Expectation Management

- Present clear results; hide technical details (p-values, etc.)

Expressions in R

Arithmetic Operators

Operation	Operator	Example	Result
Addition	+	4 + 8	12
Subtraction	-	10 - 3	7
Multiplication	*	7 * 8	56
Division	/	8 / 3	2.666667
Exponentiation	^ or **	2 ^ 5	32
Modulus	%%	5 %% 3	2
Integer division	%%/%	5 %/% 2	2
Square root	sqrt()	sqrt(25)	5

Logical Values

```
8 < 4  
3 * 2 == 5  
3 * 2 == 6  
F == FALSE  
T == TRUE
```

```
[1] FALSE  
[1] FALSE  
[1] TRUE  
[1] TRUE  
[1] TRUE
```

Using Logical Operators with Vectors

```
x <- c(1:10)  
print(x[1])  
print(x>7)  
x[(x > 7) | (x < 5)] # OR
```

```
[1] 1 2 3 4 8 9 10
```

```
x[(x > 7) & (x < 10)] # AND
```

```
[1] 8 9
```

Dates in R

System Date & Time

```
Sys.Date()  
Sys.time()  
Sys.timezone()
```

```
[1] "2017-01-13"  
[1] "2017-01-13 10:54:37 IST"  
[1] "Asia/Calcutta"
```

Formatting and Conversion

```
today <- Sys.Date()  
format(today, format = "%B %d %Y")
```

```
[1] "January 13 2017"
```

```
CustomDate <- "2016-01-13"  
class(CustomDate)  
CustDate <- as.Date(CustomDate)  
class(CustDate)  
print(CustDate)
```

```
[1] "character"  
[1] "Date"
```

Difference Between Dates

```
strDates <- c("08/15/1947", "01/26/1950")  
dates <- as.Date(strDates, "%m/%d/%Y")  
dates[2] - dates[1]
```

```
Time difference of 895 days
```

Variables

```
Var <- 50
Var
Var + 10
Var / 2
Var <- "R is a Statistical Programming Language"
Var
Var <- TRUE
Var
```

```
[1] 50
[1] 60
[1] 25
[1] "R is a Statistical Programming Language"
[1] TRUE
```

- Variables can be reassigned to different data types.

Functions: `sum()`, `min()`, `max()`

`sum()`

```
sum(1, 2, 3)
sum(1, 5, NA, na.rm = FALSE)
sum(1, 5, NA, na.rm = TRUE)
```

```
[1] 6
[1] NA
[1] 6
```

`min()` and `max()`

```
min(1, 2, 3, NA, na.rm = TRUE)
max(44, 78, 66)
```

```
[1] 1
[1] 78
```

String Functions

rep() – Repeat

```
rep("statistics", 3)
```

```
[1] "statistics" "statistics" "statistics"
```

grep() – Pattern Search

```
grep("statistical", c("R", "is", "a", "statistical", "language"), fixed = TRUE)
```

```
[1] 4
```

toupper() / tolower()

```
toupper("statistics")  
tolower("STATISTICS")
```

```
[1] "STATISTICS"  
[1] "statistics"
```

substr() – Extract Substring

```
substr("statistics", 7, 9)
```

```
[1] "tic"
```


Missing Values Treatment

Function	Description
<code>is.na(x)</code>	Returns TRUE for missing values
<code>na.omit(x)</code>	Removes missing values
<code>na.exclude(x)</code>	Removes, but keeps attributes
<code>na.fail(x)</code>	Fails if any NA present
<code>na.pass(x)</code>	Returns unchanged object

```
is.na(c(4, 5, NA))  
na.omit(c(4, 5, NA))
```

```
[1] FALSE FALSE TRUE  
[1] 4 5  
attr("na.action")  
[1] 3  
attr("class")  
[1] "omit"
```

Vectors

Creating Vectors

```
#, eval=FALSE}  
c(4, 7, 8)           # numeric
```

```
[1] 4 7 8
```

```
c("R", "is", "powerful")  # character
```

```
[1] "R"      "is"      "powerful"
```

```
c(TRUE, FALSE, TRUE)     # logical
```

```
[1] TRUE FALSE TRUE
```

Sequence Vectors

```
1:5  
seq(1, 10, 2)  
seq(10, 1, by = -2)
```

```
[1] 1 2 3 4 5  
[1] 1 3 5 7 9  
[1] 10 8 6 4 2
```

Accessing Elements

```
x <- c("R", "is", "a", "programming", "language")  
x[1]  
x[c(1, 5)]  
x[1:4]
```

```
[1] "R"  
[1] "R" "language"  
[1] "R" "is" "a" "programming"
```

Vector Names and Math

Assigning Names

```
v <- 1:5  
names(v) <- c("r", "is", "a", "programming", "language")  
v["programming"]
```

```
programming  
4
```

Vector Arithmetic

```
x <- c(4, 7, 8)  
x + 1  
x * 2  
sin(x)
```

```
[1] 5 8 9  
[1] 8 14 16  
[1] -0.7568025 0.6569866 0.9893582
```

Vector Recycling

```
c(1, 2, 3) + c(4, 5, 6, 7, 8, 9)
```

```
[1] 5 7 9 8 10 12
```

Matrices

Creating Matrices

```
matrix(1, 3, 4)           # 3x4 matrix filled with 1
a <- seq(10, 90, by = 10)
matrix(a, 3, 3)           # fill column-wise
```

	[,1]	[,2]	[,3]
[1,]	10	40	70
[2,]	20	50	80
[3,]	30	60	90

Using dim()

```
a <- 1:9
dim(a) <- c(3, 3)
a
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

Matrix Access

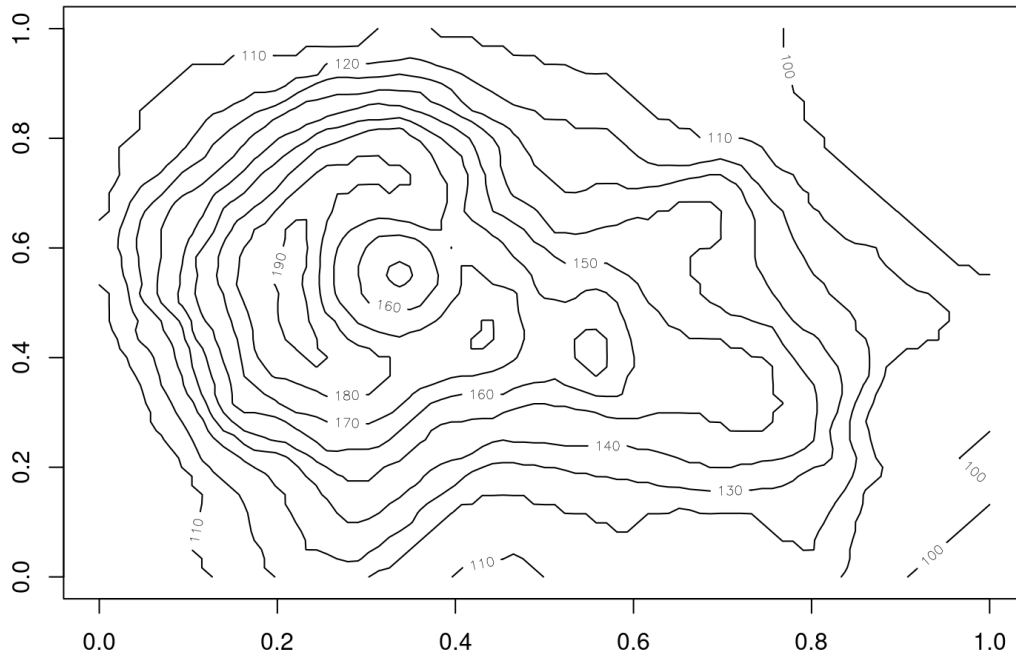
```
mat <- matrix(1:12, 3, 4)
mat
mat[2, 3]      # row 2, col 3
mat[3, ]       # row 3
mat[, 2]        # column 2
mat[, 2:3]      # columns 2 and 3
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
[1] 8
[1] 3 6 9 12
[1] 4 5 6
      [,1] [,2]
[1,]    4    7
[2,]    5    8
[3,]    6    9
```

Visualising Matrices: Contour, Perspective, Heatmap

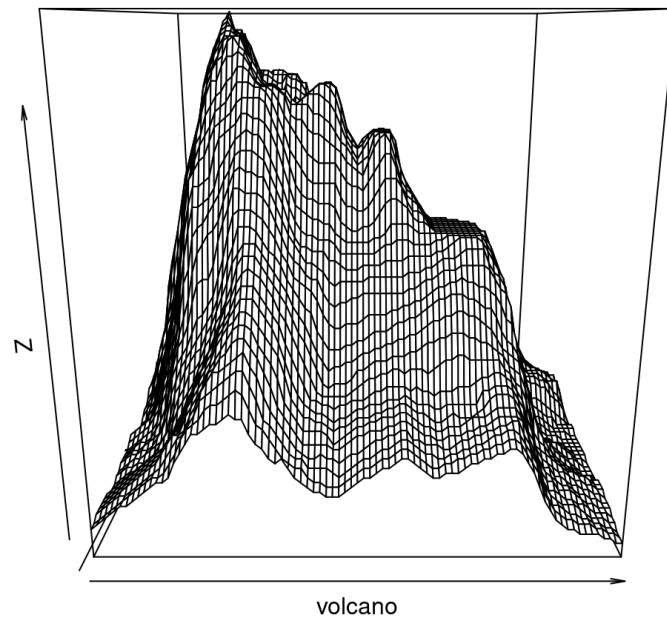
Contour Plot

```
#, eval=FALSE}  
contour(volcano)
```



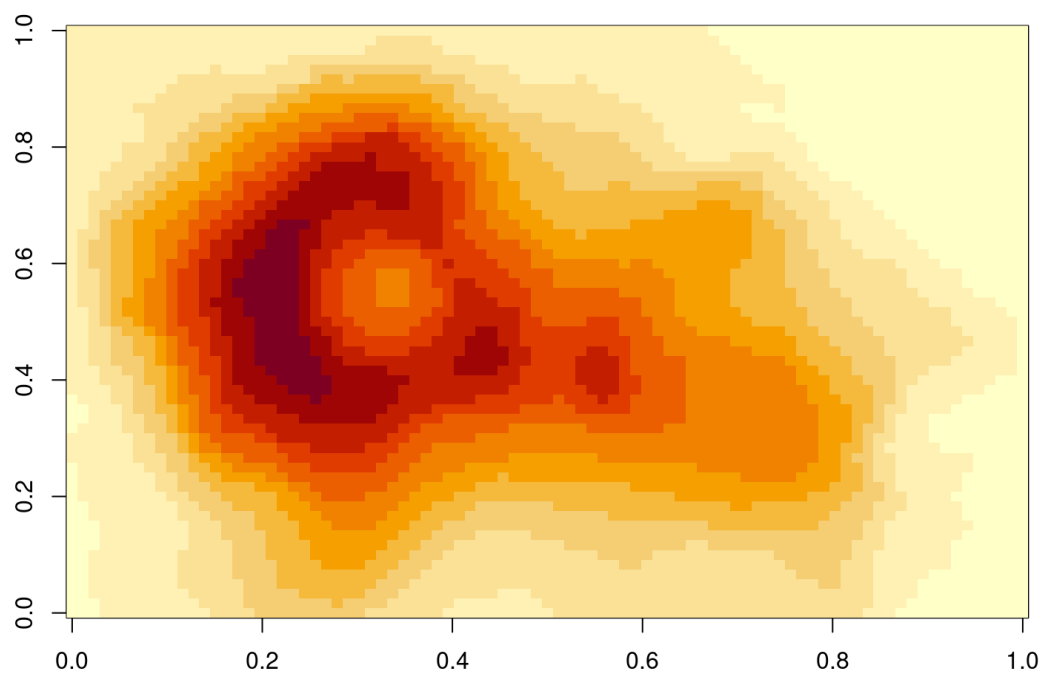
Perspective Plot

```
#, eval=FALSE}  
persp(volcano)
```



Heatmap

```
#, eval=FALSE}  
image(volcano)
```



Factors

Creating Factors

```
#, eval=FALSE}
HouseColor <- c('red', 'green', 'blue', 'yellow', 'red', 'green', 'blue', 'blue')
types <- factor(HouseColor)
types
```

```
[1] red    green  blue   yellow red    green  blue   blue
Levels: blue green red  yellow
```

```
levels(types)
```

```
[1] "blue"  "green" "red"   "yellow"
```

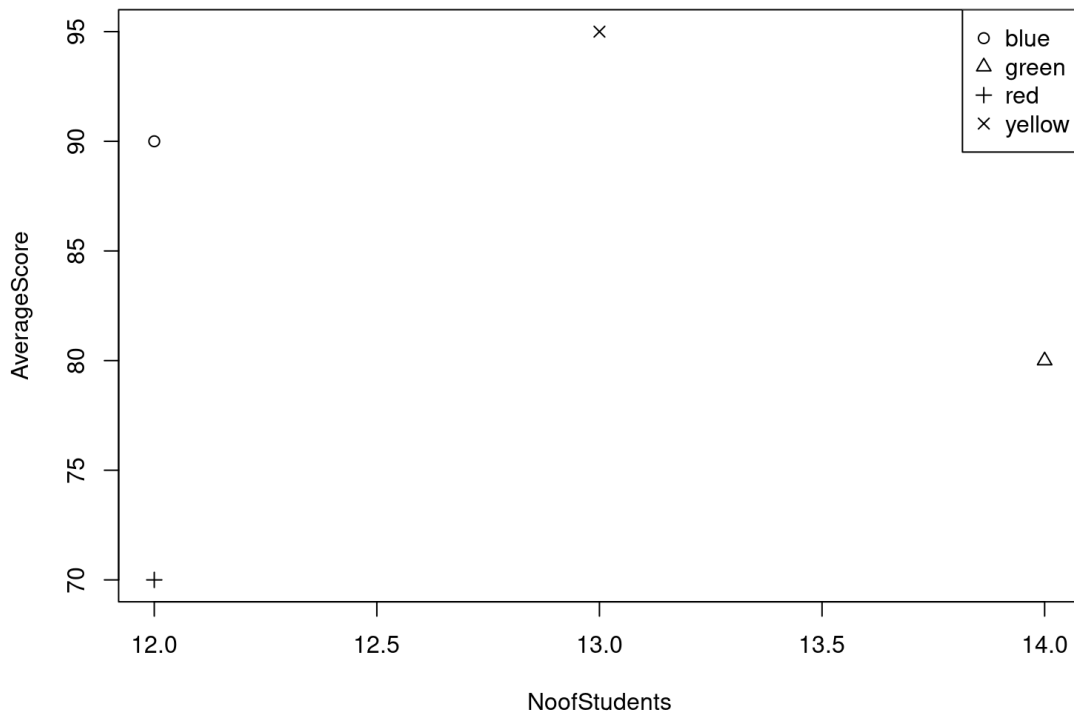
```
as.integer(types)
```

```
[1] 3 2 1 4 3 2 1 1
```

```
[1] red    green  blue   yellow red    green  blue   blue
Levels: blue green red  yellow
[1] "blue"  "green" "red"   "yellow"
[1] 3 2 1 4 3 2 1 1
```

Plotting with Factors

```
#, eval=FALSE}
NoofStudents <- c(12, 14, 12, 13)
AverageScore <- c(70, 80, 90, 95)
plot(NoofStudents, AverageScore, pch = as.integer(types))
legend("topright", levels(types), pch = 1:4)
```



Lists

Creating Lists

```
emp <- list(EmpName = "Alex", EmpUnit = "IT", EmpSal = 55000)
emp
```

```
$EmpName
[1] "Alex"

$EmpUnit
[1] "IT"

$EmpSal
[1] 55000
```

Without Tags

```
EmplList <- list("Alex", "IT", 55000)
EmplList
```

```
[[1]]
[1] "Alex"

[[2]]
[1] "IT"

[[3]]
[1] 55000
```

List Tags and Values

Retrieving Tags and Values

```
names(emp)  
unlist(emp)  
emp[["EmpName"]]
```

```
[1] "EmpName" "EmpUnit" "EmpSal"  
EmpName   EmpUnit   EmpSal  
"Alex"    "IT"      "55000"  
[1] "Alex"
```

Add / Delete Elements

```
emp$EmpDesg <- "Software Engineer"  
emp$EmpUnit <- NULL # delete
```

Size of List

```
length(emp)
```

```
[1] 3
```

Recursive List (list within a list)

```
EmplList <- list(emp, empl)
```

Exploring a Dataset

Function	Description
<code>names()</code>	Variable names
<code>summary()</code>	Summary statistics
<code>str()</code>	Internal structure
<code>head()</code>	First n rows (default 6)
<code>tail()</code>	Last n rows (default 6)
<code>class()</code>	Class of object
<code>dim()</code>	Dimensions (rows, columns)
<code>table()</code>	Frequency table of categorical variable

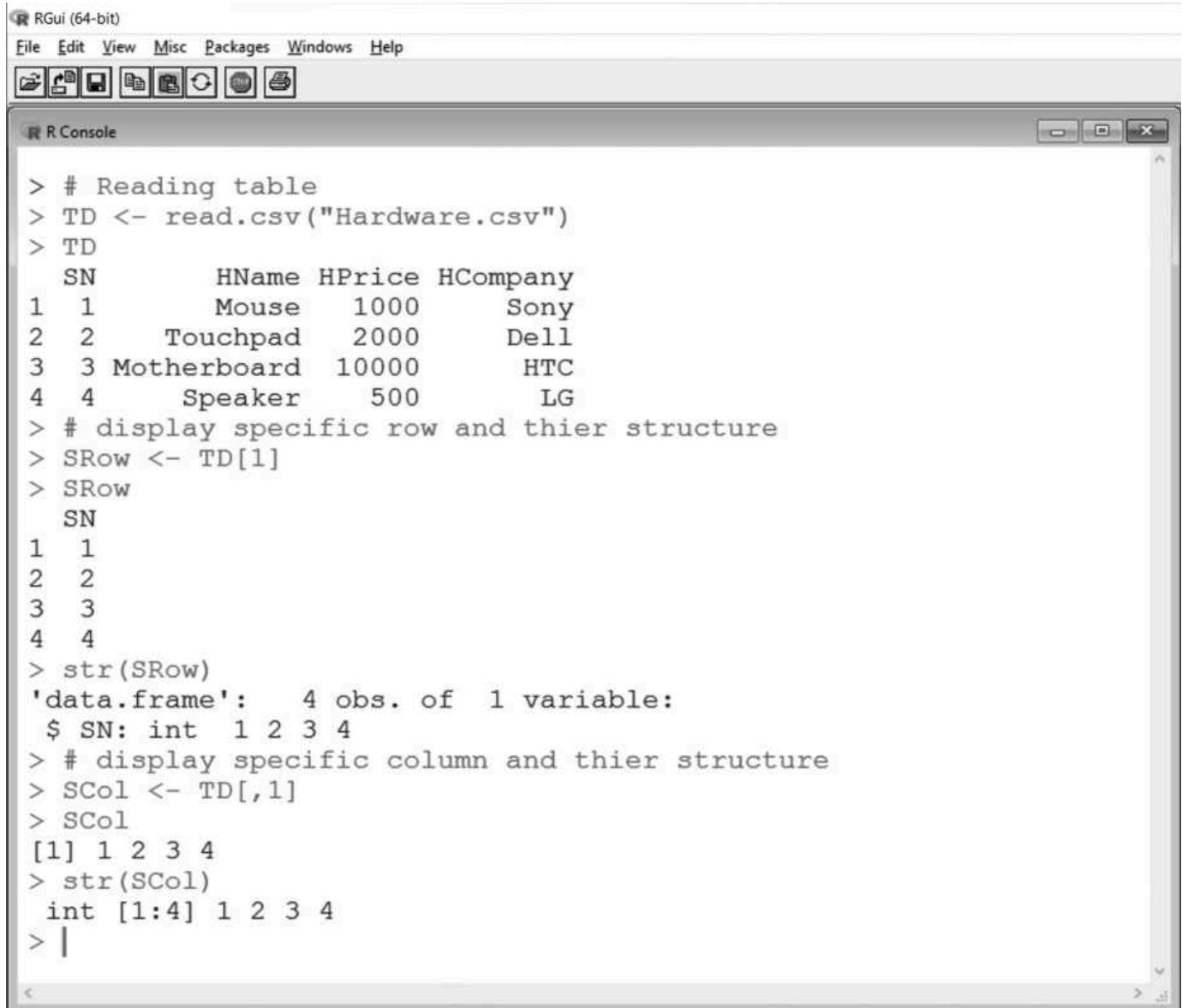
Example on Orange dataset

```
data(Orange)
names(Orange)
summary(Orange)
str(Orange)
head(Orange, 3)
tail(Orange, 3)
dim(Orange)
table(Orange$Tree)
```

Conditional Manipulation

```
# Display first row
TD[1, ]

# Display first column
TD[, 1]
```



The screenshot shows the RGui (64-bit) window with the R Console pane active. The console displays the following R code and its output:

```
> # Reading table
> TD <- read.csv("Hardware.csv")
> TD
  SN      HName HPrice HCompany
1  1      Mouse   1000      Sony
2  2  Touchpad   2000       Dell
3  3 Motherboard 10000       HTC
4  4   Speaker    500        LG
> # display specific row and thier structure
> SRow <- TD[1]
> SRow
  SN
1  1
2  2
3  3
4  4
> str(SRow)
'data.frame':  4 obs. of  1 variable:
 $ SN: int  1 2 3 4
> # display specific column and thier structure
> SCol <- TD[,1]
> SCol
[1] 1 2 3 4
> str(SCol)
 int [1:4] 1 2 3 4
> |
```

Merging Data

merge() Function

```
S <- data.frame(RN = c('A','B','E'), RM = c(10,20,30))
T <- data.frame(RN = c('A','B','C'), TM = c(10,20,30))
E <- merge(S, T) # natural join
E
```

	RN	RM	TM
1	A	10	10
2	B	20	20

Join Types

- `all = TRUE` – full outer join
- `all.x = TRUE` – left outer join
- `all.y = TRUE` – right outer join

```
E<-merge(S,T,all.y=TRUE)
E
```

	RN	RM	TM
1	A	10	10
2	B	20	20
3	C	NA	30

Aggregating and Grouping

aggregate()

```
# Assuming 'Fruit_data.csv' with columns Fruit.Name, Fruit.Price  
S <- read.csv("Fruit_data.csv")  
aggregate(S$Fruit.Price ~ S$Fruit.Name, S, mean)
```

(See Figure 3.19)

tapply()

```
tapply(S$Fruit.Price, S$Fruit.Name, sum)
```

Simple Analysis Workflow

Input

```
Fruit <- read.csv("Fruit_data.csv")
```

Describe Data Structure

```
names(Fruit)
str(Fruit)
summary(Fruit)
head(Fruit)
tail(Fruit)
```

Describe Variables

```
table(Fruit$Fruit.Name)
hist(Fruit$Fruit.Price)
boxplot(Fruit$Fruit.Price ~ Fruit$Fruit.Name)
plot(Fruit$Fruit.Price, Fruit$Fruit.Quantity)
```


Reading CSV and Spreadsheets

CSV

```
read.csv("Hardware.csv")
read.table("Hardware.csv", header = TRUE, sep = ",")
```

Excel (using xlsx package)

```
install.packages("xlsx")
library(xlsx)
read.xlsx("Softdrink.xlsx", 1) # sheet index
```

Practical Example: SampleSuperstore

```
InputData <- read.csv("d:/SampleSuperstore.csv")
GroupedInputData <- aggregate(InputData$Sales ~ InputData$Category, InputData, sum)
GroupedInputData
```

	InputData\$Category	InputData\$Sales
1	Furniture	156514.4
2	Office Supplies	132600.8
3	Technology	168638.0

Reading from Packages

`library()` and `data()`

```
library(datasets)
data()           # list all datasets
data(Orange)     # load Orange dataset
Orange
```

Reading Web Data

Packages for Web Data

- RCurL – download files, web scraping
- XML – parse XML/HTML
- WDI – World Bank data
- quantmod – Yahoo Finance

Web Scraping Example

```
library(RCurL)
library(XML)
url <- "http://example.com"
html <- getURL(url)
parsed <- htmlTreeParse(html, useInternalNodes = TRUE)
```

Reading JSON

Using rjson package

```
install.packages("rjson")
library(rjson)
output <- fromJSON(file = "d:/Jsondoc.json")
output
JSONDataFrame <- as.data.frame(output)
JSONDataFrame
```

Sample JSON data:

```
{
  "EMPID": ["1001", "2001", ...],
  "Name": ["Ricky", "Danny", ...],
  "Dept": ["IT", "Operations", ...]
}
```

Output:

	EMPID	Name	Dept
1	1001	Ricky	IT
2	2001	Danny	Operations
...			

Reading XML

Using XML package

```
install.packages("XML")
library(XML)
output <- xmlParse(file = "d:/XMLFile.xml")
rootnode <- xmlRoot(output)
rootsize <- xmlSize(rootnode) # number of nodes
xmlToDataFrame("d:/XMLFile.xml")
```

Sample XML:

```
<RECORDS>
  <EMPLOYEE>
    <EMPID>1001</EMPID>
    <EMPNAME>Merrilyn</EMPNAME>
    ...
  </EMPLOYEE>
</RECORDS>
```

Output data frame:

	EMPID	EMPNAME	SKILLS	DEPT
1	1001	Merrilyn	MongoDB	ComputerScience
2	1002	Ramya	PeopleManagement	HumanResources
...				

R GUIs for Data Input

GUI	Description
RCommander (Rcmdr)	Simple interface, many plug-ins
Rattle	Data mining GUI
RKward	Transparent front-end, cross-platform
JGR	Java GUI for R
Deducer	Menu system, works with JGR

Using R with Databases

RODBC – MS Access / SQL Server

```
library(RODBC)
conn <- odbcConnect(dsn = "servername", uid = "", pwd = "")
query <- "SELECT * FROM table"
data <- sqlQuery(conn, query)
odbcClose(conn)
```

MySQL – RMySQL

```
library(RMySQL)
conn <- dbConnect(MySQL(), user = "", password = "", dbname = "", host = "")
res <- dbSendQuery(conn, "SELECT * FROM table")
data <- fetch(res, n = -1)
dbDisconnect(conn)
```

PostgreSQL – RPostgreSQL

Similar functions: `dbConnect()`, `dbDisconnect()`.

SQLite – RSQLite

```
library(RSQLite)
conn <- dbConnect(SQLite(), dbname = "myDB.sqlite")
```

JasperDB and Pentaho

JasperReports Server

- Java library: **RevoConnectR**
- Web services: **RevoDeployR**

Pentaho Data Integration (PDI)

- **R Script Executor** – integrates R with PDI
- Allows using R scripts within ETL pipelines

Log Analysis

Reading Log Files

```
LOGS <- read.table("log.log", sep = " ", header = FALSE)
head(LOGS)
```

Analysis and Visualisation

```
summary(LOGS)
barplot(table(LOGS[, column]))
```

Save Plot

```
png("test.png", width = 1024, height = 1024)
barplot(table(LOGS[, column]))
dev.off()
```

